뭔말인교? 기술 면접 대비 — iOS·macOS 앱 + 백엔드

> 이 프로젝트의 실제 코드로 답한다. 질문마다 **30초 답** → **깊게** → **코드 위치** → **꼬리 질문** 순서. 기준: main + `v1.0/db-staged-perf-docs` (2026-10-27). 결정 근거는 ADR 번호로 단다.

1부. 구조 — 프로토콜 지향, 모듈, 라우터

Q1. "프로토콜 지향으로 짰다"는 게 구체적으로 뭔가요?

30초 답. 바깥 세계(서버·Keychain·Apple 로그인·화면 이동)를 전부 **Domain에 정의한 프로토콜** 뒤에 두고, 구현체는 Data·Platform에, 조립은 Composition 한 곳에서만 한다. 그래서 Feature는 `URLSession`도 `Keychain`도 모른다.

깊게.

프로토콜	정의	진짜 구현	테스트 구현
`TranslationRepository`	Domain	`RemoteTranslationRepository` (Data)	`DomainTesting/TranslateFakes.swift`
`AuthRepository`	Domain	`RemoteAuthRepository`	Fake
`TokenStore`	Domain	`KeychainTokenStore` (Platform)	메모리
`OAuthCodeProviding`	Domain	`AppleSignInCodeProvider`, `DevCodeProvider` (Debug)	고정 code
`AIConsentStore`	Domain	`UserDefaultsAIConsentStore`	`NoConsentStore`
`AppNavigating`	Routing	`AppRouter`	Spy
`ClientMiddleware` · `ClientTransport`	OpenAPIRuntime	Auth·Retry·Error·AIConsent / URLSession	가짜 `next`

- 상속 대신 프로토콜: 저장소가 "서버 것"과 "가짜"를 바꿔 끼우는 것 말고는 공통 구현이 없다. 공통 동작은 `extension`이나 UseCase에 둔다.
- 모든 Domain 프로토콜은 `Sendable` — Swift 6 strict에서 actor·Task 경계를 넘나든다.

코드. `Modules/Core/Domain/Sources/Repositories/*.swift`, `Modules/Core/Composition/Sources/AppSession.swift`(`live()`가 유일한 조립 지점), ADR-0016.

꼬리 질문.

- *`any TranslationRepository`와 `some`의 차이?* `any`는 존재 타입(박스 + 동적 디스패치), `some`은 컴파일 때 정해진 하나의 구체 타입. 저장 프로퍼티로 "나중에 바꿔 끼울" 의존성은 `any`가 맞다. 비용(할당·간접 호출)은 네트워크 왕복에 비하면 0에 가깝다. 뜨거운 루프(매칭 trie 등)에는 쓰지 않는다.
- *프로토콜이 너무 많아지지 않나?* 기준은 "테스트에서 바꿔 끼울 필요가 있나". `MwScale` 같은 순수 계산은 프로토콜 없이 값 타입.

Q2. Coordinator 패턴을 안 쓴 이유는?

30초 답. SwiftUI는 내비게이션이 **상태(경로 배열·시트 값)**다. 그래서 그 상태를 가진 `AppRouter` 하나면 된다. UIKit Coordinator가 하던 "누가 push하나"는 `NavigationStack(path:)` 바인딩이 대신한다.

깊게.

- 목적지는 값이다: `AppRoute.result(translationId:)`, `AppSheet.compose/result/term/settings`. 엔티티가 아니라 **id만** 싣는다 → 딥링크·상태 복원·테스트가 쉬워진다.
- 분기 규칙이 한곳에 있다: `present(_:)`는 용어·설정 시트를 **아래 시트 위에** 겹쳐 띄우고(`sheetOverSheet`, 붙여넣기 입력을 잃지 않게), 나머지는 바꿔 띄운다(붙여넣기 → 결과).
- 탭마다 경로가 따로다(`paths: [AppTab: [AppRoute]]`). 탭을 바꿔도 각 탭 위치가 남는다.
- Feature는 `AppNavigating` 프로토콜만 본다 → Feature끼리 서로 모른다.

코드. `Modules/Core/Routing/Sources/Routing.swift`, 12번 D-17.

꼬리 질문. *라우터가 거대해지면?* 기능이 늘면 탭별 라우터로 쪼갠다. 지금은 128줄. *딥링크는?* 알림 payload의 `mw.deepLink`를 `AppRoute`·`AppSheet` 값으로 바꾸는 파서 하나(ADR-0010).

Q3. 상태를 enum으로 둔 이유? (불가능한 상태 없애기)

30초 답. `isLoading`, `error`, `data`를 Bool·옵셔널 세 개로 두면 "로딩 중인데 에러도 있음" 같은 **말이 안 되는 조합**이 생긴다. enum은 한 번에 하나만 가능하다.

깊게.

- 화면 공통: `Phase<Value> = idle | loading | loaded(Value) | failed(code:)` (`FeatureKit`).
- 세션: `SessionState? = nil(확인 전) | signedOut | signedIn(UserProfile) | unverified(오프라인 시작)`. 루트 뷰가 이 값으로 `switch` 해서 로그인 화면·탭을 고른다.
- 붙여넣기: `editing | translating | failed(Reason)`, `Reason = empty | tooLong | offline | busy(seconds:) | server(message:)`. 실패 이유마다 다른 문구·버튼.
- 서버 에러 → 도메인 에러 매핑은 Data 한곳(`DomainErrorMapping.swift`). 화면은 `DomainError`만 안다.

꼬리 질문. *`switch`에 `default`를 쓰면?* 새 케이스가 생겨도 컴파일러가 안 알려 준다. 화면 상태 enum은 `default` 없이 다 쓴다(에러 매핑은 서버가 모르는 코드를 보낼 수 있어 예외).

2부. 상태 저장·삭제, 메모리, 동시성

Q4. 로그아웃할 때 지울 것을 어떻게 빠짐없이 지우나요?

30초 답. 지우는 함수가 **하나**다(`AppSession.endSession(reason:)`). 직접 로그아웃·401·시작 시 초기화가 전부 여기로 온다. 화면 상태는 뷰 수명에 묶어서, 로그인 화면으로 바뀌는 순간 같이 사라지게 했다.

깊게.

1. `state = .signedOut`을 **먼저** 바꾼다. 토큰을 지우는 사이 화면이 보낸 요청이 401을 받으면, `handleUnauthorized`의 guard(이미 signedOut이면 무시)에서 멈춘다 → "로그인이 풀렸어요"가 잘못 뜨지 않는다(QA H1).
2. Keychain 토큰 삭제 → AI 외부 전송 동의 끄기(#25).
3. 라우터·시트·화면 모델·방금 받은 결과(`prefetched`)는 `AppRootView`의 `@State`/`@ViewModelState`다. `SessionRootView`가 `.signedOut`에서 `AuthRootView`를 그리면 `AppRootView`가 트리에서 빠지고 **같이 해제**된다. 그래서 "라우터 reset 깜빡함" 같은 버그가 구조적으로 없다.
4. URL 캐시는 처음부터 없다(`urlCache = nil`) → 지울 디스크 캐시가 없다.

코드. `AppSession.swift` `endSession`, `handleUnauthorized`; `Composition.swift` `SessionRootView`.

꼬리 질문. *401이 동시에 5개 오면?* `UnauthorizedSignal`(actor)이 `pending` 플래그로 **한 번만** 알린다. 다시 로그인하면 `reset()`. AsyncStream 버퍼는 `bufferingNewest(1)`.

Q5. `[weak self]`와 `guard let self` — 언제 쓰고 언제 안 쓰나요? (순환 참조)

30초 답. **끝나지 않는 작업**(스트림 구독·타이머)은 `[weak self]` + 매번 `self?.`로, **금방 끝나는 작업**(요청 1번)은 강하게 잡아도 된다. 함정은 긴 루프 맨 위에서 `guard let self`를 하는 것 — 루프가 도는 내내 강한 참조가 되어 해제가 안 된다.

깊게 (실제 코드).

```
// AppSession.listenForUnauthorized — 앱 수명 내내 도는 루프
listener = Task { [weak self] in
    for await _ in events {
        await self?.handleUnauthorized()   // 이벤트마다 잠깐만 강하게
    }
}
```

- 잘못된 버전: `Task { [weak self] in guard let self else { return }; for await _ in events { await handleUnauthorized() } }` → `guard let self` 이후 `self`는 **강한 지역 변수**. 스트림이 안 끝나니 `AppSession`이 영원히 안 풀린다.
- `ComposeViewModel.submit()`의 `Task { [translate] in … await self.fail(error) }`는 요청 1번이 끝나면 Task도 끝난다 → 강한 참조여도 순환이 오래가지 않는다. 대신 화면이 닫히면 `cancel()`로 Task를 끊는다.

- SwiftUI `View`는 구조체라 클로저가 `self`를 잡아도 순환이 아니다. `.task {}`는 뷰가 사라지면 **자동 취소**된다.

꼬리 질문.

- *`unowned` 는?* 수명이 확실히 같거나 짧을 때만. 틀리면 크래시라 이 코드엔 없다.
- *Task가 self를 잡은 채 취소 안 되면?* 메모리는 결국 풀리지만 결과를 이미 닫힌 화면에 쓴다. 그래서 `.task(id:)` · `cancel()` · `Task.isCancelled` 검사(토스트 타이머)를 쓴다.

Q6. SwiftUI `@State` 초기화 함정이 뭔가요? 어떻게 막았나요?

30초 답. `_model = State(initialValue: ViewModel(...))`는 **부모가 다시 그릴 때마다** `init`이 불려 ViewModel을 새로 만들고 버린다. SwiftUI는 처음 것만 쓰니 화면은 멀쩡하지만, 생성 비용·부수 효과(구독 시작, 로그)가 매번 생긴다. `@ViewModelState`라는 래퍼로 만드는 식을 autoclosure로 받아 **처음 읽을 때 한 번만** 실행하게 했다.

깊게.

```
@propertyWrapper
public struct ViewModelState<Model: AnyObject & Observable>: DynamicProperty {
    @State private var box = Box()          // 빈 상자는 싸다. @State가 첫 상자만 붙잡는다
    private let make: () -> Model           // @autoclosure — 호출 전엔 아무것도 안 만든다
    public var wrappedValue: Model {
        if let model = box.model { return model }
        let model = make(); box.model = model; return model
    }
    public var projectedValue: Bindable<Model> { Bindable(wrappedValue) }  // $model.text
}
```

- 상자 안 값을 바꾸는 건 SwiftUI 상태 변경이 아니어서 "뷰 갱신 중 상태 변경" 경고가 없다.
- 같은 함정이 `@State private var router = AppRouter()`(프로퍼티 초기값도 init마다 실행)와 `SessionRootView(session: .live())`(기본 인자도 매번 실행 → Keychain·URLSession 조립)에도 있었다. 둘 다 같은 래퍼로 고쳤다.
- 화면별로 **다른 id**를 보여야 할 때(결과 시트 A → B)는 시트 `id`가 바뀌어 뷰 정체성이 새로 생기므로 새 모델이 만들어진다(의도된 동작).

코드. `Modules/Core/FeatureKit/Sources/ViewModelState.swift`, 테스트 `FeatureKitTests.swift`(만드는 식 1회, 바인딩).

꼬리 질문. *iOS 17 `@Observable` 이면 괜찮지 않나?* `@Observable`은 **다시 그리는 범위**를 줄일 뿐, `init` 재호출은 그대로다. *`.task` 에서 만들면?* 첫 프레임에 모델이 없어서 옵셔널 처리·빈 화면 한 프레임이 생긴다.

Q7. Swift 6 strict concurrency에서 실제로 부딪힌 것은?

- **actor로 경합 없애기**: `UnauthorizedSignal`은 동시 401을 직렬화하는 actor. `deliveredCount`로 테스트.
- **TaskLocal로 요청별 값 넘기기**: AI 동의를 요청마다 `AIConsent.$granted.withValue(true) { … }`로 넘긴다. 생성 클라이언트 함수 시그니처를 바꾸지 않고(계약에 헤더 파라미터가 없다) 미들웨어가 읽는다. 전역 변수였다면 동시 요청 두 개가 서로의 동의를 덮어쓴다.
- **MainActor 기본 격리**: UI 모듈(Routing·FeatureKit·Feature·Composition)은 `mainActorByDefault`. 화면 모델은 `@MainActor @Observable`. 네트워크는 `Sendable` 저장소 뒤에서 돌고 결과만 MainActor로 온다.
- **취소**: `.task(id: refreshKey)`는 키가 바뀌면 이전 로드를 취소하고 새로 시작. 토스트는 `toastTask?.cancel()` 뒤 새 타이머, 깨어나면 `Task.isCancelled` 확인.

3부. 네트워크·캐시·실시간

Q8. URLSession만 쓰면 뭐가 불편하고, 왜 Alamofire·Moya를 안 썼나요?

30초 답. URLSession만 쓰면 ① 엔드포인트 43개를 손으로 쓰고 ② 토큰·401·재시도를 매번 반복한다. Alamofire·Moya가 이걸 줄여 주지만 **여전히 손으로 쓴다**. 우리는 계약(OpenAPI)에서 클라이언트를 **생성**하고, 반복 처리는 **미들웨어 4개**로 뺐다. 계약이 바뀌면 컴파일 에러로 안다.

깊게. ADR-0022 표 그대로:

- 엔드포인트 정의 → 생성(`swift-openapi-generator`, Moya `TargetType`의 자동판)

- 인터셉터·재시도 → `ClientMiddleware` (Alamofire `RequestInterceptor` 자리): Auth / Retry(GET 503만 1회, `Retry-After` 따르고 10초 상한) / Error(공통 봉투 → `MwonmalServerError`, 바디 64KB까지만) / AIConsent
- 스텁 → `ClientTransport` 바꾸기 또는 `URLProtocol`
- 남는 Alamofire 장점: 인증서 고정(`ServerTrustManager`), `EventMonitor`.
- 전송 계층만 Alamofire로 바꾸는 안: 공식 어댑터가 없어 ~80줄 손수, 인터셉터와 미들웨어가 두 벌이 되어 "401은 누가 처리하나"가 흐려진다. 그래서 지금은 안 한다.

꼬리 질문.

- *POST는 왜 재시도 안 하나?* 해석이 두 번 저장된다. 대신 서버가 10분 안 같은 문장을 재사용(`recent_duplicate`) → 사용자가 다시 눌러도 기록은 하나.
- *200MB 녹음 백그라운드 업로드는?* 생성기 전송도 Alamofire도 background session을 지원 대상으로 두지 않는다. 그 한 경로만 `URLSession(configuration: .background)` + `uploadTask(fromFile:)`.
- *인증서 고정 안 하는 이유?* 인증서 교체 때 앱 전체가 끊기는 위험이 더 크다. ATS + 공개 CA.

Q9. 캐시는 어디에, 얼마나? 트레이드오프는?

30초 답. 원문·결과·할 일은 **기기 디스크에 0**. URL 캐시도 끄고, 쿠키도 끄고, 서버도 `no-store`를 보낸다. 대신 **같은 데이터를 두 번 받지 않는 지점**만 메모리로 메운다(방금 해석한 결과 1개, 화면 모델 1개).

깊게.

대상	정책	이유
API 응답	없음	개인 데이터. 응답 수 KB + 서버 3문 이내라 다시 받아도 싸다
방금 해석 결과	메모리 1개	붙여넣기 → 결과 시트에서 같은 요청 반복 제거. 시트 닫히면 버림
용어 상세	없음	조회수·"내 카드"가 서버 기준. 캐시하면 틀어짐. 반복 조회가 측정되면 `NSCache`(메모리만, 로그아웃 시 비움)
토큰	Keychain	유일한 영구 저장
일러스트	에셋 카탈로그 벡터	시스템이 크기별 래스터 캐시. 런타임 네트워크·디코딩 0
서버 사전	서버 메모리 5분	요청마다 134행 읽지 않게. 대가: 관리자 수정이 최대 5분 늦게 반영

- 로컬 DB(SwiftData·Realm)를 안 둔 이유: 동기화·무효화 규칙이 생기고 "같은 계정, 같은 DB" 설계와 어긋난다(15번 T-C7). 오프라인 읽기가 필요해지면 그때.

Q10. 소켓(WebSocket)은 왜 안 썼나요? 알림은?

30초 답. 서버가 먼저 보낼 이벤트가 v1.0엔 없다. 해석은 요청-응답 한 번(1~13초). v1.2 회의 녹음(최대 20분)은 **3초 폴링 + ETag**(바뀐 게 없으면 304, 바디 없음)로 받고 끝나면 APNs. WebSocket은 양방향이 필요 없고 블루·그린 배포 때 연결을 옮기는 비용, 앱이 백그라운드로 가면 끊기는 문제가 있다.

깊게.

- 되돌리는 조건: 동시 폴링 세션 200 이상 지속 → SSE(`GET /recordings/{id}/events`), WebSocket은 여전히 후보 아님(T-B2).
- 알림 payload는 `aps` + `mw.deepLink` 단일 스키마(ADR-0010). **원문·할 일 문장은 알림 본문에 넣지 않는다** — 잠금화면(남이 봄)과 Apple 서버를 지난다.
- v1.0은 원격 푸시가 없다(`user_devices`·`notifications` 표가 아직 없다, ADR-0021). macOS는 계약 v1.1 `MACOS` 등록과 함께. 워치는 iPhone 알림 포워딩.

4부. 렌더링 성능 (네이티브)

Q11. 커스텀 UI가 많은데 렌더링 비용은?

30초 답. 화면은 전부 SwiftUI 기본 뷰이고, `Canvas`·Metal·UIKit 래핑이 없다. 그림은 벡터 에셋(시스템이 크기별로 래스터화해 캐시), 모델은 `@Observable`이라 **읽은 프로퍼티만** 다시 그린다.

깊게.

- 목록은 `LazyVStack` → 보이는 행만 만든다. 다음 페이지는 마지막 행의 `.task`에서(`loadMoreIfNeeded`).

- 그림자는 행마다 오프스크린 패스가 든다. 측정 전에 `compositingGroup()` / `drawingGroup()` 을 넣지 않은 이유: 레이어 메모리가 늘고, 텍스트가 래스터가 되어 접근성 확대·다이내믹 타입에 불리하다. 프레임 드랍이 Instruments로 확인되면 그 카드만.
- 크기는 `mwScale` (기기 폭/402) 하나로 비례. 폭이 바뀔 때만 다시 계산(`onGeometryChange`).
- 키보드가 올라와 안전 영역이 커질 때 탭 바가 같이 밀리던 것(QA)은 인셋 60 이하만 받아 막았다.

꼬리 질문. *`@Observable` 인데도 리스트 전체가 다시 그려지는 경우?* 배열 자체를 바꾸면 그 배열을 읽은 뷰는 다시 그린다. 행 단위 갱신이 필요하면 행을 별도 `@Observable` 객체로 만들거나 `id` 가 안정적인 `ForEach` 를 쓴다(우리는 `Identifiable` id = 서버 id).

5부. 백엔드 성능·DB

Q12. "테이블이 너무 많다" — 계약을 안 바꾸고 어떻게 줄였나요?

30초 답. 계약·모델은 17테이블 그대로 두고, **마이그레이션만 단계화**했다. v1.0 DB는 실제로 쓰는 8개만 만든다. 나머지 9개는 대기 초안으로 두고, 기능이 열릴 때 **추가만 하는** 리비전으로 붙인다 → 기존 데이터에 영향 0.

깊게.

- 지연 테이블을 쓰는 코드는 `schema.has("cards")` 같은 게이트로 막고, 막혔을 때 계약상 맞는 값(회의 없음 → `MEETING_NOT_FOUND` , 개수 0)을 돌려준다.
- 없는 표를 가리키는 외래 키(`translations.meeting_id → meetings`)는 v1.0에 만들지 않는다. MySQL은 없는 표를 참조하는 FK를 만들 수 없다.
- 테스트가 매번 "head + 대기 초안 = 계약 17테이블, 차이 0"과 되돌리기를 확인한다 → 1년 뒤 기능을 열 때 초안이 계약과 어긋나 있는 일을 막는다.
- 테이블 합치기·JSON 컬럼은 계약 위반이고, 합친 표는 인덱스·잠금 범위가 커진다.

코드. `services/api/app/core/schema.py` , `migrations/staged/*.py` , `tests/test_staged_migrations.py` , ADR-0021.

Q13. 백엔드를 빠르게 하려고 실제로 뭘 했나요? (측정 기반)

30초 답. 지연은 대부분 **DB 왕복 수**가 정한다. 요청당 SQL 문 수를 기준으로 삼아 POST 11→6, 상세 5→3, 프로젝트 2+6N→3, 용어 6→3으로 줄였다. 프로파일러로 찾은 JWT 키 재파싱도 없앴다. 결과적으로 같은 기계에서 처리량이 1.3~3배.

깊게.

문제	해결	수치
저장 뒤 같은 데이터를 다시 읽음	매칭 때 이미 아는 용어 이름·쉬운 말로 응답을 메모리에서 조립. **POST 응답 == 바로 GET 응답**을 테스트로 고정	11→6문, p50 15.3→5.5ms
프로젝트 목록 N+1(프로젝트마다 6문)	표마다 `GROUP BY project_id … WHERE project_id IN (…)` 한 문	74→227 rps
용어 조회: refresh로 다시 읽기	조회수 UPDATE 먼저 → 같은 트랜잭션에서 SELECT(내가 쓴 최신 값을 본다)	6→3문
PyJWT에 PEM 바이트 → 요청마다 키 파싱	키 객체를 캐시	요청당 ~0.5ms
연결 풀 기본값	10+10, 대기 15초(AI 호출 12초 동안 연결을 쥐므로), 1800초 재생성	5프로세스 × 20 = 100 < 151

측정으로 기각한 것 (면접에서 제일 좋은 이야기).

- `count(*) over()` 로 목록과 전체 개수를 한 문에 → 왕복은 줄지만 DB가 **LIMIT 전에 조건에 맞는 행을 전부** 읽는다. `(user_id, created_at)` 인덱스로 20행에서 멈추던 게 멈추지 않아 **110→218ms로 느려졌다**. 따로 세는 COUNT(인덱스만 훑음)로 되돌렸다. 프로젝트처럼 사용자당 몇 행인 곳에만 쓴다.
- orjson: 프로파일에서 직렬화가 상위에 없었다 → 의존성만 늘어서 안 넣었다.

꼬리 질문.

- *`pool_pre_ping` 은 왕복 하나 더인데 왜 켜 두나?* DB 재시작·장애 조치 뒤 죽은 연결로 첫 요청이 500 나는 게 0.3ms보다 비싸다.
- *SQLite로 잰 수치를 믿나?* "요청당 문 수"는 DB와 무관하게 같고, 그게 운영 지연을 가장 크게 바꾼다. 절대 수치는 MySQL 스테이징에서 다시 잰다. SQLite는 쓰기 잠금이 하나라 동시 POST p95는 의미 없다고 ADR에 적었다.
- *다음 병목은?* 해석 요청이 AI 호출 동안 DB 연결을 쥔다 → 읽기 → 연결 반납 → AI → 짧은 쓰기 트랜잭션으로 나눈다(v1.1).

Q14. 동시성 버그를 실제로 만난 적은? (MySQL 교착)

30초 답. 첫 해석 때 "기본 프로젝트가 없으면 만든다"를 동시에 8개 보내면 MySQL 1213(데드락)이 났다. 갭 락 두 개가 서로를 기다린 것. **사용자 행을 `SELECT … FOR UPDATE` 로 먼저 잠가** 같은 사용자의 생성 요청을 줄 세웠다.

깊게. "없으면 INSERT"는 유니크 인덱스 갭 락(S) → INSERT 의도 락(X) 순서라 두 트랜잭션이 서로 막는다. 재시도 루프보다 **잠글 대상을 하나로** 하는 게 결정적이다. 테스트: `test_default_project_race.py` (MySQL에서 8 동시).

Q15. 개인정보는 어떻게 지키나요? (설계 원칙)

- 원문·받아쓰기·오디오는 기기·우리 서버 밖으로 나가지 않는다. 외부 AI(Gemini)는 **옵트인 헤더 + 서버 설정 + 키**가 모두 있을 때만(ADR-0008·0020).
- 로그에 원문 0: DB 오류의 바인드 값 숨김(`hide_parameters`), 500은 예외 타입·파일:줄만, 카나리 테스트로 원문이 로그에 0회인지 확인.
- 앱: URL 캐시 0, 알림 본문에 원문 0, 로그아웃·401 시 토큰·동의 삭제.

6부. 토스 과제식 짧은 문제

#	질문	핵심 답
1	같은 버튼을 두 번 빨리 누르면 해석이 두 번 가나?	VM이 `state == .translating` 이면 무시(guard). 서버도 10분 안 같은 문장 재사용 → 이중 방어
2	해석 중 시트를 닫으면?	`cancel()` 로 Task 취소, 상태는 `editing` 으로. 이미 서버에 도착했으면 저장은 된다(사용자는 기록에서 본다)
3	401이 여러 요청에서 동시에 오면 로그인 화면이 여러 번?	actor 신호가 1번만. 상태를 먼저 바꿔 두 번째부터는 guard에서 멈춤
4	할 일 체크가 서버 저장에 실패하면?	먼저 화면을 바꾸고(낙관적 갱신), 실패하면 그 행과 남은 개수를 옛 값으로 되돌리고 토스트(`HomeViewModel.toggle`). 401이면 재로그인 흐름으로
5	페이지네이션 중 새 기록이 생기면 중복?	오프셋 방식이라 한 칸 밀릴 수 있다. v1.0은 목록 맨 위 새로고침으로 해결, 무한 스크롤이 길어지면 커서(`created_at,id`) 방식
6	아이폰 SE ~ Pro Max에서 같은 비율?	Figma 402pt 기준 `mwScale = 폭/402` . 텍스트·여백·아이콘이 같이 커진다. 단위 테스트는 CI에서 iPhone SE(iOS 17.0)·최신 iPhone 둘 다, E2E는 iPhone 17 Pro
7	오프라인으로 앱을 켜면?	토큰은 있으면 `unverified` 로 들어가 탭은 보이고, 요청은 바로 실패(`waitsForConnectivity=false`) → "연결 없음" 카드. 다시 닿으면 `reloadProfile` 로 회복
8	다크 모드·다이내믹 타입?	색은 토큰(`MwColor` , 색 38개 중 35개가 라이트·다크 쌍). 글꼴은 `mwFont` 가 기기 배율 × `@ScaledMetric` (시스템 글자 크기)를 곱한다
9	계약이 바뀌면 앱·서버 중 누가 먼저 깨지나?	앱은 생성 코드 컴파일 에러, 서버는 schemathesis 계약 검사(1,083 케이스) CI 실패. 둘 다 머지 전
10	"테이블 줄여라" vs "계약 고정" 충돌은?	계약은 그대로, 마이그레이션만 단계화. 테스트로 계약 1:1 유지

부록. 숫자 한 장

- 서버 테스트 310개 통과(MySQL 6개는 CI), 계약 검사 1,083 케이스 통과, 사전 134개.
- 요청당 SQL: POST 6 · 상세 3 · 목록 2 · 할 일 2 · 프로젝트 3 · 용어 3.
- 앱: 모듈 12개(Core 8 + Feature 4), E2E 1개(화면 15장 스크린숏), 최소 OS iOS 17 / macOS 14.
- 결정 기록: ADR 22개, 트레이드오프 기록 15번 문서.